
1. What happens when a user clicks “Search”?

When a user enters a location or keyword and clicks the Search button, we send that query to Firestore and fetch all PGs that match the keyword. Then we display the results dynamically on the Search Results page using React.

2. How is the filter implemented on the search results page?

We apply filters (like gender, mess type, price range) using state variables in React. When a user changes a filter, we update the query or filter the already fetched PGs locally based on those values.

3. How does the “Submit PG” form work – what happens after clicking submit?

When the owner fills in the PG form and clicks Submit, we collect all data using form inputs, upload the image to Firebase Storage, get the image URL, and then save all the PG info into Firestore under the owner’s ID.

4. How do you fetch and display PG listings from Firestore?

We use Firebase’s Firestore SDK to query data from the `pgListings` collection. Then we map through the documents and display each PG card using React components.

5. Can you describe how real-time chat works in your system?

Each chat is stored in Firestore under a unique chat ID based on user-owner pair. When a user sends a message, it’s stored as a document in a subcollection. Firestore syncs it in real time, so the owner sees the message instantly.

6. How do you prevent users from submitting blank or incorrect forms?

We added frontend validation using JavaScript before submission. We check if fields are empty, prices are valid, and images are selected. We also show alert messages if anything is wrong.

7. How do you display only PGs that match the selected filters?

We either filter the data locally using JavaScript after fetching from Firestore, or apply compound Firestore queries (when possible) to fetch only the relevant results based on filter selections.

8. What is the flow from logging in to sending a message?

First, the user logs in via Firebase Auth. Once logged in, they can go to any PG listing and click "Chat." This opens the chat screen, where they can send a message, which is stored in Firestore in real-time.

9. How does the app differentiate between a "user" and an "owner"?

When someone signs up, they choose their role (user or owner), and we store that role in Firestore with their UID. Based on the role, we show different components and restrict access to pages like "Submit PG" or "Search Rooms."

10. What will a new owner see after signing up and logging in?

After logging in, the owner is taken to their dashboard. They can update their profile, submit PGs, view their PG listings, and chat with users who message them.

11. How did you structure Firestore collections and documents?

We created collections like `pgListings`, `users`, and `chats`.

- Each PG listing is a document inside `pgListings` with fields like title, rent, ownerId, imageURL, etc.
 - Each user has a document in the `users` collection with role info.
 - For chat, we used a `chats` collection, where each chat has a subcollection called `messages`.
-

12. How did you enforce role-based restrictions (both in frontend and Firestore rules)?

In the frontend, we check the user's role (user or owner) from Firestore and show/hide features accordingly.

In Firestore, we use security rules to ensure only owners can write to `pgListings`, and only logged-in users can read messages.

13. How does `useEffect` help in your PG listing or chat components?

`useEffect` lets us run code when the component loads or updates.

We use it to:

- Fetch PG listings when the page loads
 - Load chat messages in real-time
 - Scroll to the latest message
-

14. How are images stored and retrieved securely using Firebase Storage?

When an owner uploads an image, we upload it to Firebase Storage. It returns a secure download URL, which we save in Firestore. We show this image using the URL. Access is controlled by Firebase rules.

15. How is authentication handled on both the frontend and backend?

On the frontend, users log in using Firebase Auth (email or Google). Firebase stores their session in the browser.

On the backend (Firestore), we check their `uid` in rules to allow or deny data access.

16. What are the Firestore read/write rules you wrote?

Examples:

- Only authenticated users can read data
- Only owners can write to their own PG listings
- Only users involved in a chat can read/write messages

Rules are written using `request.auth.uid == resource.data.ownerId` or similar checks.

17. How do you ensure that only owners can edit their own listings?

In Firestore rules, we check if the user's UID matches the `ownerId` in the PG document. If it doesn't match, the write request is denied.

18. What are the edge cases you tested for in chat or room submission?

- Blank form submission
 - Duplicate PG names
 - Image not selected
 - No internet during chat or submit
 - User trying to access owner-only pages
 - Unauthorized write to Firestore
-

19. How do you fetch dynamic route data (e.g., `/details/:id`) in React?

We use React Router's `useParams()` to get the room ID from the URL, then fetch the PG details from Firestore using that ID.

20. How is the user redirected after login? How is session maintained?

After login, we use `useNavigate()` to redirect the user to the homepage or dashboard. Firebase maintains the login session in local storage, so users stay logged in until they log out manually.

21. How would you implement an admin panel to moderate listings?

We'd create a new role called "admin" and assign it in the users collection. The admin dashboard would list all PGs, and allow actions like approve, delete, or flag. Firestore rules would allow only admin UIDs to perform those actions.

22. How do you plan to prevent spam messages in chat?

We can:

- Add a cooldown (delay between messages)

- Limit message length
 - Block repeated or duplicate messages
 - Add reporting and moderation features in the future
-

23. What are the security concerns of using Firebase in production?

Some key concerns are:

- Exposing database rules to public if not secured
- Allowing unauthorized writes or reads
- Misconfigured Storage access

We avoided this by writing strict rules and not exposing sensitive user data.

24. How would you handle failed image uploads or internet loss during submit?

We'd show a message saying "Upload failed" and prevent form submission. We can retry the upload or store it temporarily and try again. Firebase also supports error handling on image upload events.

25. How scalable is your current Firebase setup? What limits exist?

Firebase scales automatically for small to medium apps.

Limits include:

- 50K reads/day (free tier)
- 1MB per document
- 1 write/second per document

For a small student-level app, it's more than enough.

26. How would you integrate Razorpay or Google Maps in your project?

For Razorpay: Use Razorpay JS SDK and trigger payment on booking confirmation.

For Maps: Use Google Maps JavaScript API and embed the map on the room listing or search result page.

27. How would you handle user feedback and ratings without abuse?

We'd allow reviews only after booking is confirmed. Users can leave 1 review per room. Admins can hide abusive reviews. We'd also store review metadata like user ID and timestamp for moderation.

28. How does Firestore differ from a traditional SQL database in use-case?

Firestore is NoSQL and stores data in documents (JSON-like), not rows and tables. It's more flexible for dynamic, nested data like chat, room listings, or user profiles. SQL is better for complex joins and structured schemas.

29. If a user bookmarks PGs, where and how do you store that data?

We store bookmarked PG IDs in a subcollection under the user's Firestore document. When they open their dashboard, we fetch and display those PGs using the stored IDs.

30. What happens if a user reloads the page while inside a room details page?

Nothing breaks. We use dynamic routing (`/details/:id`), so on reload, the `useParams()` grabs the ID from the URL again, and fetches the PG data from Firestore.

31. What is the difference between controlled and uncontrolled components in React?

Controlled components are form elements where the value is controlled by React state (e.g., `value={name}` with `useState`).

Uncontrolled components store their own value internally and we access them using `ref` .

We used **controlled components** for all forms to track input and validation easily.

32. How do you handle multiple inputs in a form using React?

We use a single state object like `formData` and update fields using the `onChange` handler.

Example:

```
setFormData({...formData, [e.target.name]: e.target.value})
```

33. Why is `useEffect` called with dependencies?

The dependency array (`[]` or `[value]`) tells React when to re-run the effect.

- `[]` runs once on load
- `[someValue]` runs when that value changes

It helps us control when data is fetched or when to update components.

34. How is conditional rendering used in login, roles, or submit pages?

We check user status or role and render elements accordingly.

Examples:

- If not logged in → show "Login" button
 - If role is `owner` → show "Submit PG"
 - If `loading` is true → show a spinner
-

35. How is React Router's `useParams` or `useNavigate` used in your app?

- `useParams` is used to get dynamic route values like PG ID from the URL (e.g., `/details/:id`)
 - `useNavigate` is used to programmatically go to another page after actions like login or submit
-

36. How did you handle error messages and success feedback?

We used `useState` to store success or error messages, and displayed them using conditional rendering. For example:

```
{error && <p className="text-red-500">{error}</p>}
```

37. What are the advantages of component-based architecture?

- Reusability: Same UI component can be used multiple times
- Maintainability: Easier to manage code in small parts

- Scalability: We can build complex UIs by combining components
 - Testing and debugging becomes easier
-

38. What is props drilling and did you experience it?

Props drilling is when we pass data through many components unnecessarily.

We avoided it by using local state in each component or by restructuring components cleanly.

39. How do you secure PG listing edits so only the owner can change them?

In Firestore rules, we check if `request.auth.uid` matches the `ownerId` of the listing. If not, we deny the write.

We also hide the edit button in the frontend unless the logged-in user is the owner.

40. How do you secure Firestore collections from public access?

We wrote Firebase security rules to allow access only:

- If the user is authenticated (`request.auth != null`)
 - And only if the user's UID matches the document's owner field
- This protects chat, PGs, and user profiles from outsiders.
-

41. Can users modify Firestore data from the browser dev tools?

They can try, but it won't work if Firestore rules are correctly set.

Even if someone edits data in the browser using JavaScript, **Firestore backend checks every read and write**. If rules don't allow it, the operation is rejected.

42. What steps would you take if someone reverse-engineered your frontend?

If someone tries to manipulate frontend code:

- They can see UI but can't bypass backend rules
- We avoid storing secrets in the frontend

- Firebase rules and role checks protect the backend

For more protection, we can add obfuscation and verification in future.

43. How is chat data protected – can other users access it?

No. Chat data is stored in subcollections tied to a user-owner pair.

In Firestore rules, we ensure that **only the two participants (by UID)** can read/write those messages.

44. How are passwords stored in Firebase?

Passwords are **hashed and stored securely** by Firebase Auth.

We never access or see the passwords ourselves – Firebase handles all encryption and login logic.

45. Is your login system vulnerable to brute force or phishing?

Firebase protects against brute force with built-in rate limiting.

However, users can still fall for phishing if they share credentials. We can prevent that by using **Google login**, which is more secure and passwordless.

46. How did you test your system before final submission?

We tested:

- Login and registration for both roles
- Submitting rooms and checking they appear in search
- Real-time chat
- Routing and redirects
- Edge cases like submitting empty forms

Also tested on mobile and different browsers.

47. What tools or browser features did you use for debugging?

We used:

- Chrome DevTools to inspect console logs, network requests
 - Firebase Console to monitor data changes
 - React Developer Tools to check component state and props
-

48. How would you identify and fix a bug if a PG listing fails to upload?

First, we'd check the console for errors (e.g., permission denied, file too large).

Then we'd:

- Check Firestore rules
 - Add try-catch blocks around form submission
 - Log the error to understand the cause
 - Reproduce the issue locally
-

49. How do you test responsiveness and design consistency?

We used:

- Chrome DevTools responsive mode
 - Tested on multiple screen sizes (mobile, tablet, desktop)
 - Tailwind's responsive classes like `md:` , `lg:` to control layout
 - Made sure elements don't overflow or break
-

50. What tools or methods would you use for app performance monitoring?

In future, we can use:

- **Firestore Performance Monitoring**
 - Chrome Lighthouse for load time and optimization
 - Console logs and analytics to track user actions
- Right now, we check loading speed manually and optimize images/code.
-

51. What if Firebase raises its pricing or becomes unavailable?

We would:

- Shift to another backend like **MongoDB + Node.js** or **Supabase**
 - Backup Firestore data and migrate it to a new database
 - Use a custom backend hosted on platforms like **Render, Railway, or Heroku**
- Also, Firebase has a generous free tier, which is good enough for MVPs and student projects.
-

52. How would you migrate data to MongoDB or another system?

We'd:

- Export Firestore data using Firebase Admin SDK or JSON export
- Write a script to convert the data format to MongoDB (documents and collections)
- Use MongoDB's import tool (`mongoimport`) to load data into a new database

Then, update our frontend to connect using APIs or a new backend.

53. Can this system be turned into a mobile app? How?

Yes. We can convert it into a mobile app using:

- **React Native**: reuse most of our frontend logic
- Or **Flutter**, but that would need rewriting UI

The backend (Firebase) stays the same — we just rebuild the frontend UI for mobile.

54. How would you handle multi-language/localization features?

We would:

- Use a library like **react-i18next** to manage multiple languages
- Store all text content in separate JSON files (e.g., `en.json` , `bn.json`)
- Detect or allow users to select their language

Then dynamically render UI based on their preference.

55. Can you generate a report of all PG listings by owner ID?

Yes. We can:

- Query Firestore for listings where `ownerId == currentUserID`
- Format that data into a table or export to PDF using libraries like `jsPDF` or `react-pdf`

This can be shown in the owner dashboard as a "Download Report" option.
